

Flex、Grid布局

2025年10月10日

目 录

01 ▶ 概述

02 ▶ Flex布局

03 ▶ Grid布局

01 概述

CSS3带来一个新的布局方案-Flex布局。因为要考虑兼容性，所以Flex布局早期主要用在移动端，后来随着工程化工具的支持，PC端也开始普及Flex布局。

Flex布局使用起来非常顺手。例如，之前要实现一个简单的居中布局，还要考虑子元素是块级元素还是行内元素。采用Flex布局只需要设置父元素即可，可以无视子元素类型。

1、CSS弹性盒子技巧——<https://css-tricks.com/snippets/css/a-guide-to-flexbox/>

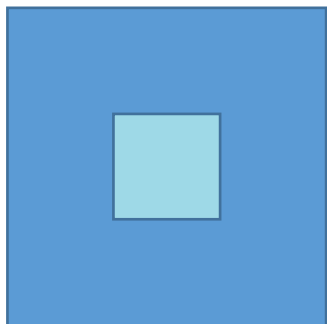
以视觉吸引人的方式解释弹性盒子所有内容的文章

2、弹性盒子青蛙游戏——<https://flexboxfroggy.com/>

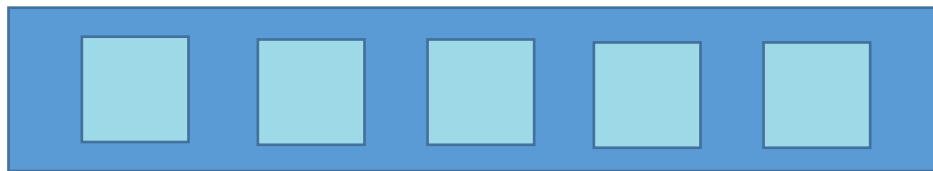
学习和更好地了解弹性盒子基础知识的教育游戏

02 Flex布局

弹性盒子布局可以解决如下问题



垂直居中



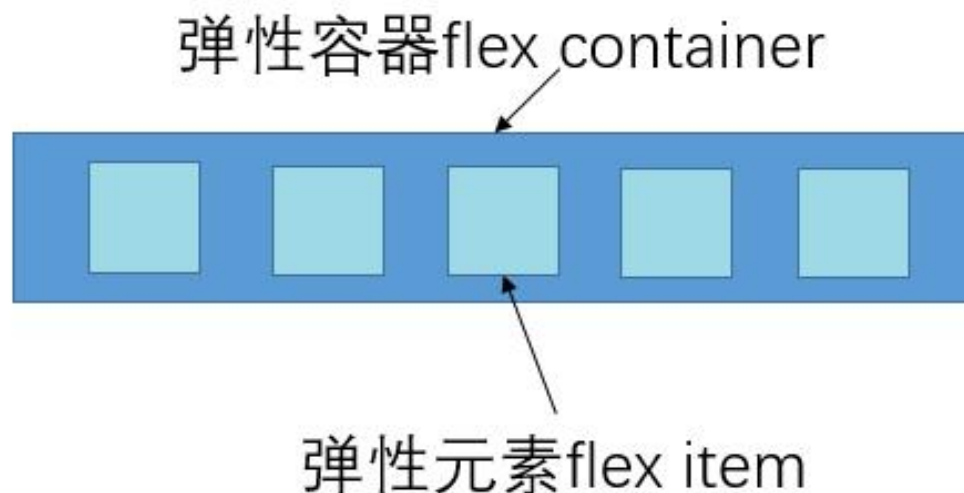
间隙的平均分配



自动占据剩余空间

02 Flex布局

采用 Flex 布局的元素，称为 Flex 容器（flex container），简称“容器”。它的所有子元素自动成为容器成员，称为 Flex 元素（flex item），简称“元素”。



Flex布局原理：就是通过给父盒子添加flex属性，来控制子盒子的位置和排列方式。弹性元素也可以作为新的弹性容器，嵌套一些新的其他的弹性元素。

02 Flex布局

display:flex

flex-direction

flex-wrap

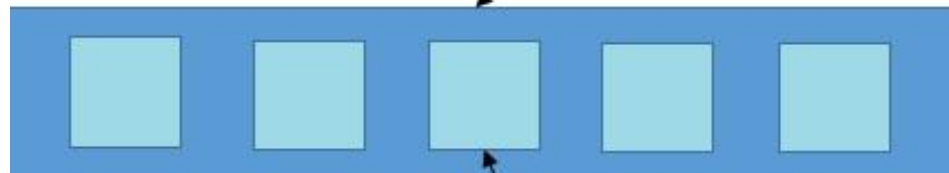
flex-flow

justify-content

align-items

align-content

弹性容器flex container



弹性元素flex item

flex-grow

flex-shrink

flex-basis

flex

order

align-self

flex-direction: 设置按行还是按列进行布局

flex-wrap: 设置子元素是否换行

justify-content: 设置弹性元素在主轴上的对齐方式

align-items: 设置弹性元素在侧轴上的对齐方式（单行）

align-content: 设置弹性元素在侧轴上的对齐方式（多行）

flex-direction设置主轴的方向

在 flex 布局中，是分为主轴和侧轴两个方向，同样的叫法有：行和列、x 轴和y轴。

- 默认主轴方向就是 x 轴方向，水平向右
- 默认侧轴方向就是 y 轴方向，水平向下

flex-direction属性的可选值有以下4个：

属性值	说明
row	默认值从左到右
row-reverse	从右到左
column	从上到下
column-reverse	从下到上

02 Flex布局

弹性容器样式



row



row-reverse



column



column-reverse

justify-content 用于设置弹性元素在主轴上的对齐方式

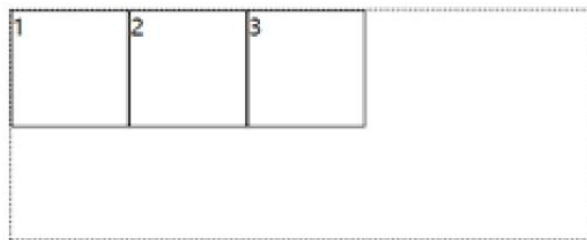
注意：使用这个属性之前一定要确定好主轴是哪个
该属性的可选值如下：

属性值	说明
flex-start	默认值 从头部开始 如果主轴是x轴，则从左到右
flex-end	从尾部开始排列
center	在主轴居中对齐（如果主轴是x轴则 水平居中）
space-around	平分剩余空间
space-between	先两边贴边 再平分剩余空间（重要）

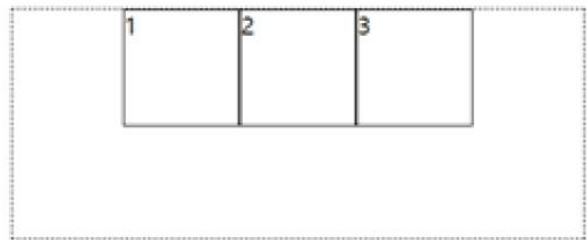
02 Flex布局

弹性容器样式

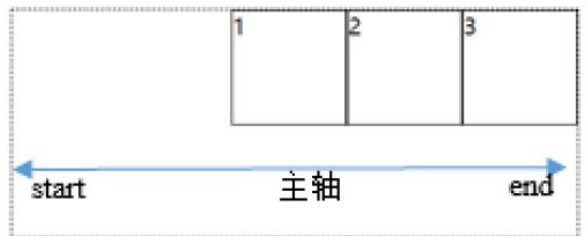
justify-content: flex-start



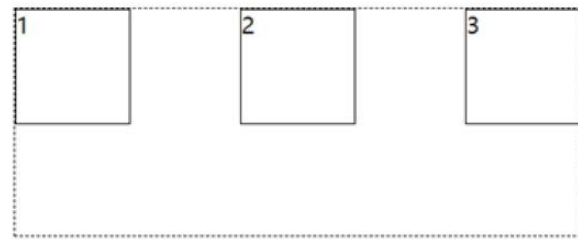
justify-content: center



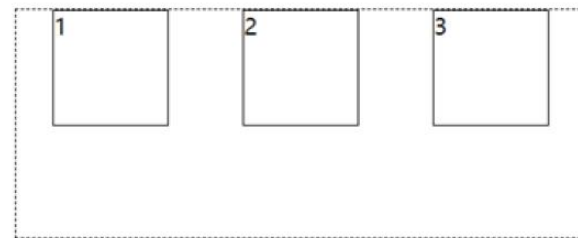
justify-content: flex-end



justify-content: space-between



justify-content: space-around



space-between表示子元素两端对齐，中间平均分配

space-around表示在子元素的两侧平均分配

除了主轴方向的元素对齐，还可以用`align-items`属性设置交叉轴方向的元素对齐。`align-items`属性的可选值如下：

1. `flex-start`: 从上到下
2. `flex-end`: 从下到上
3. `center`: 居中对齐
4. `baseline`: 基线对齐
5. `stretch`: 填满整个高度（默认）

`baseline`是指按照文字的基线对齐，因为一个容器内不同文字的大小可能不同，高度也就会不同，采用基线对齐就可以按照文字的最低处对齐，这样有利于文字排版。

`stretch`表示填满整个父元素的高度。

当元素在一个方向放不下时，应该如何展示？是否需要换行？

容器元素可以使用flex-wrap属性设置，该属性可选值如下：

1. nowrap：不换行（默认）
2. wrap：换行，第一行在上
3. wrap-reverse：换行，第一行在下

当一个轴的元素放不下时，默认不换行，Flex容器会将元素的宽度等比例压缩，使其排列到一行。在一般情况下，如果需要换行，将flex-wrap属性设置为wrap即可，超出元素会自动换到下一行。



align-content 设置多行元素在容器中的整体对齐方式
只有一行或者一列，这个属性不起作用

属性值	说明
flex-start	默认值在侧轴的头部开始排列
flex-end	在侧轴的尾部开始排列
center	在侧轴中间显示
space-around	子项在侧轴平分剩余空间
space-between	子项在侧轴先分布在两头，再平分剩余空间
stretch	设置子项元素高度平分父元素高度

align-content 和 align-items 区别

- align-items 适用于单行情况下， 只有上对齐、下对齐、居中和 拉伸
- align-content 适应于换行（多行）的情况下（单行情况下无效）， 可以设置 上对齐、下对齐、居中、拉伸以及平均分配剩余空间等属性值。
- 单行用 align-items，多行用 align-content

- **flex-grow** 用于设置弹性元素被拉大的比例
- **flex-shrink** 用于设置弹性元素被压缩的比例
- **flex-basis** 用于设置元素在主轴上的初始尺寸，即元素放入容器前的默认大小，可能被压缩，放入容器后，根据元素是否需要拉伸或压缩计算实际长度，其优先级高于width属性。
- **align-self** 控制子项自己在侧轴的排列方式
- **order** 定义子项的排列顺序（前后顺序）

flex-grow	数值	在主轴方向有剩余空间时，空间分配给元素的比例
flex-shrink	数值	在主轴方向空间不足时，元素应收缩的尺寸比例
flex-basis	em/rem/px n%	元素的基准主尺寸

flex	flex-grow flex-shrink flex-basis	前三个属性的简记法
------	----------------------------------	-----------

flex默认值分别为0 1 auto; (即不放大，会缩小)

注：当 flex-wrap 为 wrap 或 wrap-reverse 时 flex-shrink 不会生效

```
.container {  
  display: flex;  
  flex-direction: row;  
}  
.a {  
  flex: 1 1 100px;  
}  
.b {  
  flex: 2 1 100px;  
}  
.c {  
  flex: 1 2 100px;  
}
```

当 .container 的 width 为 500px 时:

由于 $500 > (100+100+100)$, 各元素 grow

grow 的单位 $(500-(100+100+100))/(1+2+1)=50(\text{px})$

.a 分配到 $50*1+100=150(\text{px})$

.b 分配到 $50*2+100=200(\text{px})$

.c 分配到 $50*1+100=150(\text{px})$

当 .container 的 width 为 200px 时:

由于 $200 < (100+100+100)$, 各元素 shrink

shrink 的单位 $((100+100+100)-200)/(1+1+2) = 25(\text{px})$

.a 分配到 $100-25*1=75(\text{px})$

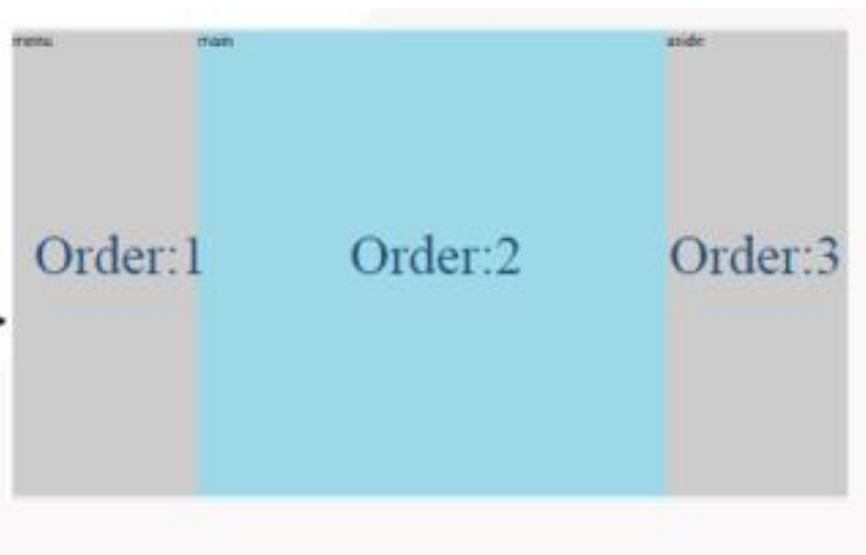
.b 分配到 $100-25*1=75(\text{px})$

.c 分配到 $100-25*2=50(\text{px})$

order属性

能够改变子元素在容器内的位置。不会影响到源顺序（即 DOM 树里元素的顺序），所有子元素默认order值为0，数值越小排名越靠前。相同 order 值的子元素按源顺序显示。也可以给 order 设置负值使它们比值为 0 的元素排得更前面。

```
<div class="flex-container">  
  <div class="flex-item">main</div>  
  <div class="flex-item">menu</div>  
  <div class="flex-item">aside</div>  
</div>
```



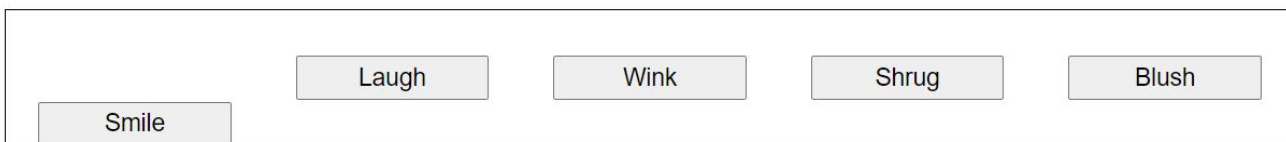
align-self属性 控制子项自己在侧轴上的对齐方式

align-self 属性允许单个项目有与其他项目不一样的对齐方式，可覆盖 align-items 属性，align-items 设置的是全部属性，align-self可以设置在单个子元素上。

align-self默认值为 auto，表示继承父元素的 align-items 属性，如果没有父元素，则等同于 stretch。

align-self: auto | flex-start | flex-end | center | baseline | stretch

可覆盖 align-items 属性



```
<div>
  <button>Smile</button>
  <button>Laugh</button>
  <button>Wink</button>
  <button>Shrug</button>
  <button>Blush</button>
</div>
```

```
div {
  display: flex;
  align-items: center;
  justify-content: space-around;
}
button:first-child {
  align-self: flex-end;
}
```

03 Grid布局

Grid（网格布局）是一种用于二维布局的CSS技术，可以非常灵活和强大方式来排列、对齐和分配容器内元素的空间。Grid布局是CSS布局模型的一种，旨在提供一种更加直观和有效的布局方法，可以代替传统的Flexbox布局模型。

网格容器（grid container）

`display: grid;`

`grid-template-columns`

`grid-template-rows`

`grid-template-areas`

`grid-gap`

`justify-items`

`align-items`

`justify-content`

`align-content`

网格元素（grid item）

`grid-item-*`

`grid-row-*`

`grid-area`

`justify-self`

`align-self`

简写为`place-self`

简写为`place-items`

简写为`place-content`

`grid-template-columns` 和 `grid-template-rows` 划分网格的行和列，

取值：`px`、`%`、`auto`、`fr`、`repeat()` 函数

<code>grid-template-columns: 1fr 1fr;</code>	→		表示分为两列，每列占比是相等的
<code>grid-template-columns : 1fr 2fr;</code>	→		
<code>grid-template-columns : 100px 1fr 2fr;</code>	→		fr可以与一般的长度单位混合使用。
<code>grid-template-columns : repeat (3, 1fr);</code>	→		

`grid-template-columns: repeat(3, 1fr);`

第一个传入 `repeat` 函数的值3，表明了后续列宽的配置要重复多少次，第二个值1fr表示需要重复的配置，这个配置可以具有多个长度设定。

例如`repeat(2, 2fr 1fr)`，相当于填入了 `2fr 1fr 2fr 1fr`。

使用 grid-template-areas 属性放置元素

grid-area 属性可以对网格元素进行命名。命名的网格元素可以在属性中使用这些名字作为一个区域。

grid-template-areas属性的使用规则如下：

- 需要填满网格的每个格子；
- 对于某个横跨多个格子的元素，重复写上那个元素grid-area属性定义的区域名字；
- 所有名字只能出现在一个连续的区域，不能在不同的位置出现；
- 一个连续的区域必须是一个矩形；
- 使用” . “符号，让一个格子留空

```
.container {  
  display: grid;  
  grid-template-areas:  
    "header header"  
    "sidebar content"  
    "footer footer";  
  grid-template-columns: 1fr 3fr;  
  gap: 20px;  
}
```

```
header {  
  grid-area: header;  
}  
  
article {  
  grid-area: content;  
}
```

```
aside {  
  grid-area: sidebar;  
}  
  
footer {  
  grid-area: footer;  
}
```

03 Grid布局

网格容器样式

练习



justify-items、align-items、place-items 属性

用于设置网格元素在网格中的位置

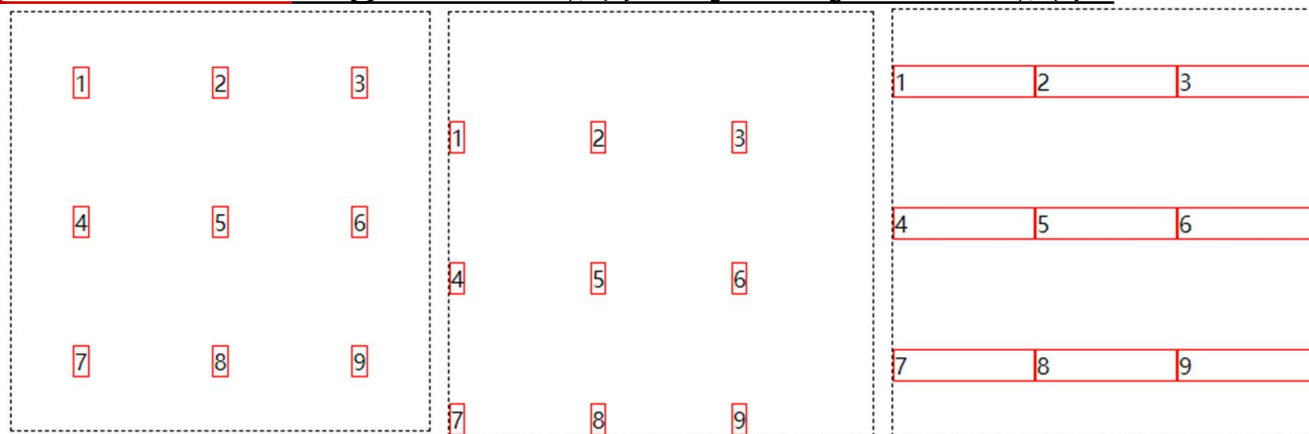
(1) justify-items: 单元格内容的水平位置

justify-items : start | end | center | 默认 stretch;

(2) align-items: 单元格内容的垂直位置

align-items : start | end | center | 默认 stretch;

place-items: align-items 属性 justify-items 属性



justify-items: center;
align-items: center;

justify-items: start;
align-items: end;

justify-items: stretch;
align-items: center;

03 Grid布局

justify-content、align-content、place-content 属性

用于设置整个内容区域在容器里面的位置。

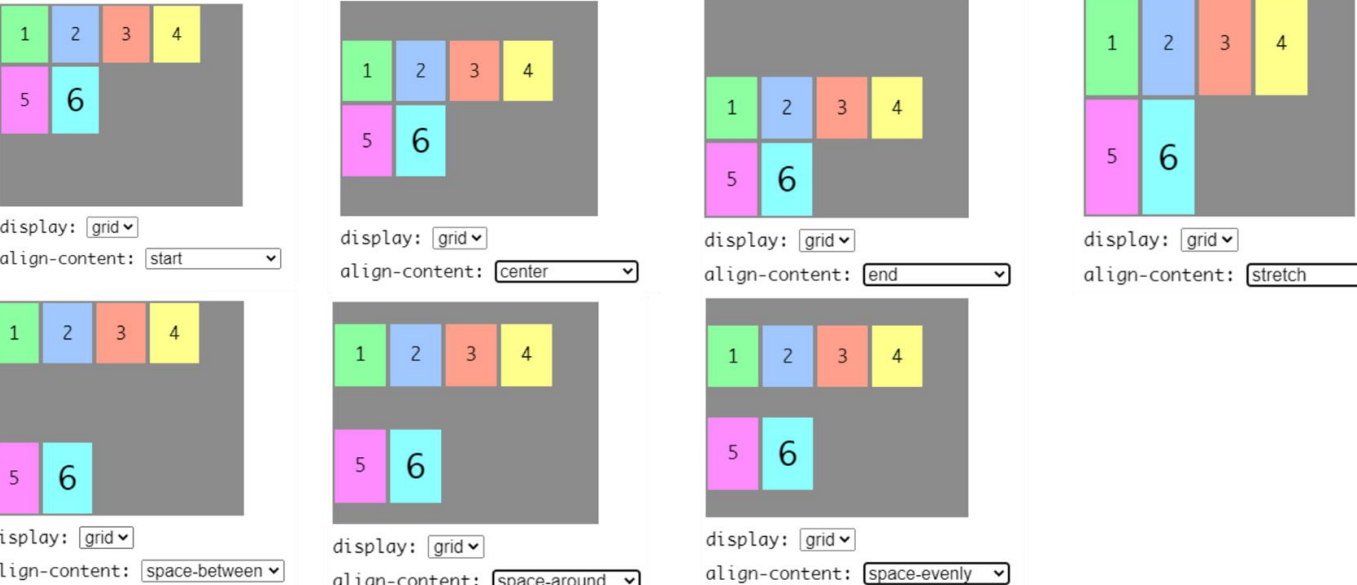
(1) justify-content: 定义整个内容区域在容器里面的水平位置（左中右）。

justify-content : start | end | center | stretch | space-around | space-between | space-evenly;

(2) align-content: 定义整个内容区域在容器里面的垂直位置（上中下）

align-content : start | end | center | stretch | space-around | space-between | space-evenly;

place-content: align-content属性 justify-content属性 ;



display:
align-content:

display:
align-content:

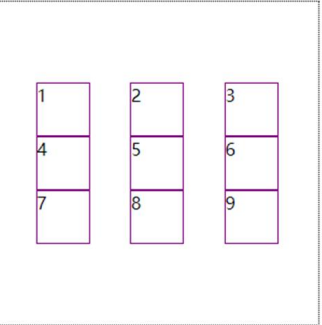
display:
align-content:

display:
align-content:

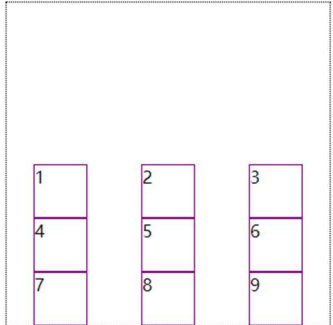
display:
align-content:

display:
align-content:

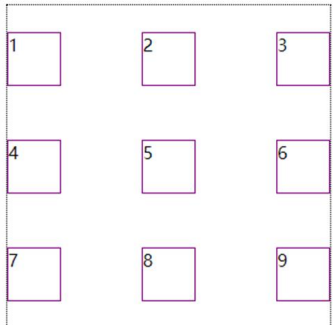
display:
align-content:



justify-content: space-evenly;
align-content: center;



justify-content: space-around;
align-content: end;



justify-content: space-between;
align-content: space-around;

基于线的元素放置

把网格元素按照网格线进行放置，每条网格线从1开始自动编号，元素的位置按照指定的网格线定义。通过以下属性来指定从哪条线开始到哪条线结束：

grid-column-start、grid-column-end、grid-row-start、grid-row-end

也可以用缩写：grid-column、grid-row（开始与结束的线的序号要使用/符号分开）

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 3fr;  
  gap: 20px;  
}
```

```
/* 基于线的元素放置 */  
header {  
  grid-column: 1 / 3;  
  grid-row: 1;  
}  
  
article {  
  grid-column: 2;  
  grid-row: 2;  
}
```

```
aside {  
  grid-column: 1;  
  grid-row: 2;  
}  
  
footer {  
  grid-column: 1 / 3;  
  grid-row: 3;  
}
```

03 Grid布局

网格元素样式



基于线的元素放置

还可以使用 `grid-area` 一次性指定所有的行/列序号： `/ / /` ，也就是先指定开始坐标的行/列序号，再指定结束坐标的行/列序号。

例如： `2/2/3/3`：行起始位置/列起始位置/行结束位置/列结束位置

`/* 使用grid-area一次性指定所有行列序号 */`

```
header{
  grid-area: 1/1/2/3;
}
article{
  grid-area:2/2/3/3;
}
aside{
  grid-area: 2/1/3/2;
}
footer{
  grid-area: 3/1/4/3;
}
```

03 Grid布局

`justify-self`、`align-self`、`place-self` 属性

单个网格元素的对齐方式

(1) `justify-self`属性：单元格内容的水平位置，用法同`justify-items`属性。

`justify-self`: `start` | `end` | `center` | `stretch` (水平拉伸)

(2) `align-self`属性：单元格内容的垂直位置，用法同`align-items`属性。

`align-self`: `start` | `end` | `center` | `stretch` (垂直拉伸)

`place-self`: `align-self`属性 `justify-self`属性;



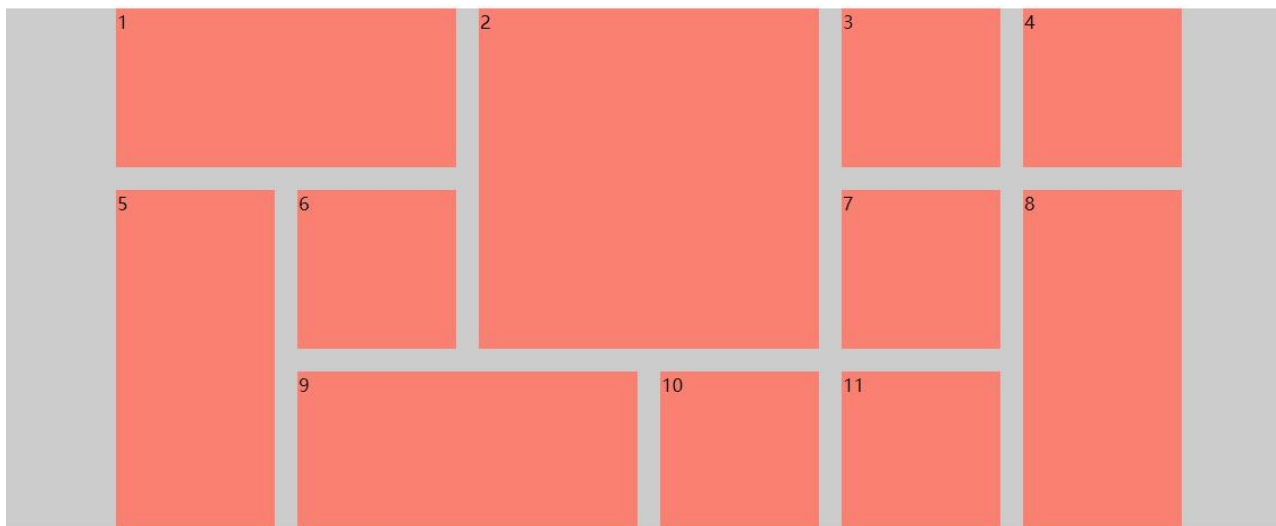
```
span:nth-child(2) {  
  justify-self: start;  
}
```

```
span:nth-child(3) {  
  justify-self: center;  
}
```

```
span:nth-child(4) {  
  justify-self: end;  
}
```

03 Grid布局

案例



```
<div class= “grid-container” >  
  <div class= “grid-item” >1</div>  
  .....  
  <div class= “grid-item” >11</div>  
</div>
```

```
.grid-container{  
  display:grid;  
  justify-content:center;  
  align-content:center;  
  grid-template-columns:repeat(6,140px);  
  grid-template-rows:repeat(3,140px);  
  gap:20px;  
}  
  
.grid-item:nth-child(1){  
  grid-column: 1 / span 2;  
  grid-row: 1 / 2;  
}  
  
.grid-item:nth-child(2){  
  grid-column: 3 / span 2;  
  grid-row: 1 / span 2;  
}
```

网格元素样式

grid-row: 1 / span 2;意思是网格元素从第1条水平线开始，到跨越2个网格线的位置结束